DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2

Name: R Sai Hasnika

REG NO: 192424178

COURSE CODE: DSA0308

COURSE: Natural Language Processing

Introduction to Word-Level and Syntactic Analysis

Natural language processing converts unstructured language into representations that a computer can process. Word-level analysis begins by deciding what counts as a token. Syntactic analysis then studies how tokens combine and what grammatical roles they perform. These stages are connected: a tokenisation decision changes the vocabulary, the contexts observed by an n-gram model, and the input units presented to a part-of-speech tagger. A small preprocessing decision can therefore influence every later measurement.

The assignment follows an evidence-driven workflow: predict, design, implement, measure, analyse, evaluate, diagnose, redesign and validate. A good report should not merely list definitions. It should explain why a design was selected, what evidence would confirm it, and what limitations remain. The same corpus split and preprocessing policy should be used when comparing alternatives so that the comparison is fair.

Experimental Design and Reproducibility

A reproducible NLP experiment records the corpus name, version, licence or access route, preprocessing steps, train/test split, random seed, vocabulary policy, model parameters and evaluation metric. Without these details, another person cannot determine whether a reported difference is caused by the model or by a change in data preparation. The split must be performed before fitting vocabulary statistics whenever test leakage is possible.

For each experiment, save both a machine-readable result file and a short interpretation. A CSV should contain one row per model, data condition and training proportion. Keep raw counts separate from derived ratios. For example, store token count N and vocabulary size V, then calculate V/N. This makes it possible to verify the calculation and update the report if the corpus changes.

Q1 – What Counts as a Word? Tokenisation and Word Statistics

Problem Context – Customer-Review Corpus

A telecom company collects customer reviews such as:

“Can’t connect!! Net speed 4G/5G isn’t consistent :(”

“Won’t recharge via e-wallet”

“Rs.499 plan is value-for-money”

“It is the-best-network, no? ok ok”

“Super service da, semma speed”

Every NLP component downstream (n-gram models, taggers) depends on how these strings are split into words.

What Is a Token?

A token is an individual unit produced by a tokenisation procedure. A type is a distinct token form in the resulting vocabulary. Tokenisation is not a universal operation with one inevitable answer. It is a design decision shaped by the task, language, corpus and downstream model. In a review such as “Can’t connect!!”, one policy may retain “Can’t” as one token, while another may split it into “Ca” and “n’t”. Both outputs are possible, but they represent different assumptions.

For a language model, consistency and predictive usefulness are central. For a tagger, preserving grammatical units such as clitics may be important because they can carry syntactic information. The policy must be documented and applied consistently to training, development and test data. Otherwise, apparent model errors may actually be preprocessing mismatches.

Tokenization Criteria

The first criterion is case handling. Case folding maps forms such as “Net” and “net” to a common lowercase representation. This reduces vocabulary size but loses information about sentence starts, proper nouns and

acronyms. The second criterion is punctuation. Sentence-final punctuation may be removed from lexical tokens but retained as a separate token if punctuation is relevant to syntax or sentence boundaries.

The third criterion concerns contractions and clitics. A contraction can be retained as one orthographic token or split into components such as “do” and “n’t”. The fourth criterion concerns hyphenated compounds. A form such as “value-for-money” may be one token or three lexical components. The fifth criterion concerns numbers and currency: “Rs.499” may be retained, split into currency and number, or normalised to a generic price marker. The sixth criterion concerns emoticons and repeated words, which may express sentiment or emphasis.

Decision	Possible treatment	Effect
Case	Preserve or lowercase	Changes type identity
Punctuation	Detach, retain or discard	Changes token boundaries
Contractions	Keep whole or split	Changes grammatical units
Compounds	Keep whole or split	Changes N and V

Applying the Criteria to Telecom Reviews

Consider “Can’t connect!! Net speed 4G/5G isn’t consistent :(”. Under a refined policy, lowercase alphabetic forms, retain contractions as single tokens, preserve the slash-separated network expression as one token, treat emoticons as tokens, and detach repeated punctuation. The resulting sequence could be: “can’t”, “connect”, “net”, “speed”, “4g/5g”, “isn’t”, “consistent”, “:(”. This gives eight tokens. The exact count depends on whether the contraction is split and whether 4G/5G is one unit.

For “Won’t recharge via e-wallet”, retaining the contraction and hyphenated compound gives four tokens: “won’t”, “recharge”, “via”, “e-wallet”. If the compound is split, the count becomes five. For “Rs.499 plan is value-for-

money", a policy that treats the currency-price string and compound as single units gives four tokens. These are reasoned predictions under a stated policy, not universal counts.

Decision	Possible treatment	Effect
Case	Preserve or lowercase	Changes type identity
Punctuation	Detach, retain or discard	Changes token boundaries
Contractions	Keep whole or split	Changes grammatical units
Compounds	Keep whole or split	Changes N and V

Tokenisation of Remaining Reviews

For "It is the-best-network, no? ok ok", retain the hyphenated compound, separate punctuation and preserve repeated words. The sequence is "it", "is", "the-best-network", "no", "ok", "ok", giving six lexical tokens if punctuation is not counted. The repeated "ok" contributes two tokens but only one type. For "Super service da, semma speed", retain the regional-language words as ordinary tokens and detach the comma. This gives five tokens: "super", "service", "da", "semma", "speed".

The examples show why token counts and type counts differ. Repeated occurrences increase N but do not necessarily increase V. Case folding may merge types; splitting contractions and compounds may increase both N and V. Punctuation handling may increase N if punctuation marks become separate tokens. The report should state whether punctuation tokens are included in lexical statistics.

Decision	Possible treatment	Effect
Case	Preserve or lowercase	Changes type identity
Punctuation	Detach, retain or discard	Changes token boundaries
Contractions	Keep whole or split	Changes grammatical

		units
Compounds	Keep whole or split	Changes N and V

An Ambiguous Engineered String

An additional ambiguous example is “Call me @ 5:30pm—Rs. 499/- ok? :)”. The string contains a social-media marker, a time, a currency expression, a dash, a question mark and an emoticon. A policy that preserves time and currency expressions as single units may produce “call”, “me”, “@”, “5:30pm”, “rs.499/-”, “ok”, “:)”. A different policy may split the time into “5:30” and “pm”, split the currency marker from the amount, and treat “@” as punctuation.

This example exposes a limitation in fixed rules: a symbol may be meaningful in one context and decorative in another. A robust tokenizer may need context-sensitive rules or a domain-specific normalisation layer. The ambiguity should be reported rather than hidden. The engineered case is useful because it tests whether the chosen criteria are precise enough to be implemented consistently.

Three Tokenisation Policies

The naive whitespace policy splits only on whitespace. It is simple and fast, but punctuation remains attached to words. Thus “connect!!” and “connect” become different types, and “review,” differs from “review”. A punctuation-aware policy separates punctuation marks and may normalise case. It usually reduces accidental distinctions but requires rules for apostrophes, decimals, abbreviations and emoticons.

A refined policy adds domain-aware rules for contractions, hyphenated compounds, currency, network labels and emoticons. It should not be assumed that the refined policy always has the smallest vocabulary. Splitting a compound can increase the number of distinct component types, while case folding can reduce the vocabulary. The measured V therefore reflects the combined effects of all decisions.

Decision	Possible treatment	Effect
Case	Preserve or lowercase	Changes type identity
Punctuation	Detach, retain or discard	Changes token boundaries
Contractions	Keep whole or split	Changes grammatical units
Compounds	Keep whole or split	Changes N and V

Word Statistics: N, V, TTR and Hapax

N is the total number of token occurrences. V is the number of distinct token types. The type-token ratio is TTR = V/N. A hapax legomenon is a type that occurs exactly once in the corpus. Hapax count is a useful indicator of vocabulary tail size and lexical sparsity. It is sensitive to corpus size: a larger sample generally contains more rare forms.

A low TTR does not automatically mean that a corpus is linguistically simple. TTR tends to decrease as corpus length grows because common words repeat. When comparing corpora of different sizes, a length-normalised measure such as moving-average TTR may be more informative. For the assignment, report the required raw TTR but interpret it in light of N.

Decision	Possible treatment	Effect
Case	Preserve or lowercase	Changes type identity
Punctuation	Detach, retain or discard	Changes token boundaries
Contractions	Keep whole or split	Changes grammatical units
Compounds	Keep whole or split	Changes N and V

Rank–Frequency Distribution and Zipf's Law

Zipf's law describes an approximate inverse relationship between a word's frequency and its rank. If words are ranked from most frequent to least frequent, frequency often follows f(r) approximately proportional to r raised to a negative exponent. Taking logarithms gives a roughly linear relation: log f(r) = constant − s log r. The slope of a log–log rank-frequency plot is therefore approximately −s.

To estimate the slope, count token frequencies, sort them in descending order, assign ranks beginning at one, and fit a linear regression to log(rank) and log(frequency). Exclude zero-frequency values because their logarithm is undefined. The estimated slope depends on the fitting range and corpus. Report the range used and do not treat Zipf's law as an exact law for every rank.

Experimental Procedure and Interpretation

Select a cited corpus containing at least 20,000 tokens. A review corpus is appropriate if the goal is to model customer language; a balanced general corpus may be useful for comparison. Apply the three policies to the same text. For each policy, calculate N, V, TTR, hapax count and the ten most frequent tokens. Save the token sequence or frequency dictionary so that the results can be independently checked.

The expected qualitative pattern is that whitespace splitting preserves many punctuation-attached variants, while punctuation-aware processing may merge or separate forms differently. Case folding often reduces V. Splitting contractions and compounds can increase N and potentially V. These are predictions only; the actual direction and magnitude must be established from the chosen corpus.

Decision	Possible treatment	Effect
Case	Preserve or lowercase	Changes type identity
Punctuation	Detach, retain or	Changes token

	discard	boundaries
Contractions	Keep whole or split	Changes grammatical units
Compounds	Keep whole or split	Changes N and V

Evidence Table and Design Decision

Use a table with columns for policy, N, V, TTR, hapax count and top ten tokens. Add the corpus identifier and preprocessing version to the caption. Do not insert invented counts. A blank template is preferable to unsupported numbers, because the rubric specifically asks for counted experimental evidence. The rank-frequency plot should have logarithmic axes and a fitted trend line.

For a language model, a refined policy that preserves meaningful domain expressions while normalising superficial variation is a defensible choice. For a tagger, splitting grammatical clitics may be more useful if the tagset represents them separately. The best policy depends on the downstream task, not on token count alone. A single tokenizer need not be optimal for every NLP component.

Q2 – Unsmoothed N-grams and the Sparsity Problem

Problem Context – Predictive Typing for a Campus Helpdesk

A helpdesk keyboard suggests the next word from the previous one or two words. A maximum-likelihood (unsmoothed) n-gram model is trained on past tickets.

The Chain Rule

The probability of a sentence consisting of words w1 through wn can be expanded using the chain rule: P(w1,…,wn) = product over i of P(wi | w1,…,w(i−1)). This identity is exact. It does not assume that words depend only on nearby context. The practical difficulty is estimating a separate conditional probability for every possible history, because many histories occur rarely or never.

An n-gram model approximates the full history with a limited context. A unigram ignores previous words. A bigram conditions on the immediately previous word. A trigram conditions on the previous two words. A four-gram conditions on the previous three. Increasing n can capture more local structure but fragments the training data across more contexts.

Maximum-Likelihood N-gram Estimates

For a unigram, the maximum-likelihood estimate is P(w) = C(w)/N, where C(w) is the number of occurrences of w and N is the total number of training tokens. For a bigram, P(wi | wi−1) = C(wi−1,wi)/C(wi−1). For a trigram, P(wi | wi−2,wi−1) = C(wi−2,wi−1,wi)/C(wi−2,wi−1). The same count ratio extends to higher orders.

These estimates assign probability zero to an event whose numerator count is zero. A sentence containing even one zero-probability n-gram receives probability zero under a product model. This is a serious limitation for predictive typing because a new or uncommon phrase can be rejected completely even when it is linguistically plausible.

Measure	Meaning	How to report
Distinct n-grams	Unique observed sequences	Count by order
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count
Perplexity	Exponentiated cross-entropy	Held-out value

Predicting Sparsity for n = 1 to 4

Before running the experiment, predict that the number of distinct observed n-gram types generally increases as n increases, although the exact count is bounded by the number of possible windows. The proportion of test n-grams unseen in training is expected to rise with n because longer sequences are less likely to repeat exactly. The number of zero-probability test sentences is also expected to rise because a single unseen n-gram is sufficient to make a sentence probability zero.

These are hypotheses, not guaranteed numerical outcomes. Repeated templates can make higher-order n-grams common in a narrow domain. A dataset containing a repeated phrase may show lower unseen rates than a diverse corpus. The edge case is therefore valuable: it demonstrates that data composition can alter the apparent relationship between n and sparsity.

Measure	Meaning	How to report
Distinct n-grams	Unique observed sequences	Count by order
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count

Perplexity	Exponentiated cross-entropy	Held-out value

Three Contrasting Datasets

The in-domain dataset should contain campus helpdesk tickets, for example password-reset requests, Wi-Fi problems, timetable queries and account-access issues. The out-of-domain dataset may contain news text. The edge-case dataset can be constructed from very short messages and repeated templates, such as “wifi not working” or “reset my password”. Keep the source and selection procedure documented.

Use a fixed 80/20 split and a fixed random seed. If the data contains near-duplicate tickets, consider splitting by ticket or conversation rather than by individual sentence, because sentence-level random splitting may place nearly identical text in both training and test sets. This can inflate apparent generalisation.

N-gram Implementation Plan

Tokenise each sentence using one consistent policy, add sentence-boundary markers, and count all n-grams for n from one to four. Store counts in a counter or dictionary. For each n, report the number of distinct training n-grams, the percentage of test n-gram occurrences not observed in training, and the number of test sentences containing at least one unseen n-gram.

Generate sample text by selecting a starting context and repeatedly sampling a next word according to the estimated conditional distribution. Without smoothing, generation can stop when a context has no observed continuation. Record the stopping rule and random seed. Generated sentences are qualitative illustrations; they should not be used as a substitute for held-out evaluation.

Reading the Results

A model begins to memorise when its higher-order contexts become specific to training examples and provide little reliable coverage of new text. Evidence includes a rapidly increasing number of distinct n-grams, a high unseen test rate, and many zero-probability sentences. The divergence between training and test coverage is caused by limited data, varied wording, domain shift and the combinatorial growth of possible sequences.

With only 50 MB of ticket data and no smoothing, a bigram is a reasonable initial choice because it captures immediate local context while retaining more repeated observations than a high-order model. This is a design recommendation, not a measured conclusion. If the measured trigram coverage is sufficiently high and the test set is tightly matched, a trigram may be justified. The final choice must cite the actual counts.

Q3 – Smoothing: Add-One, Backoff and Deleted Interpolation

Problem Context – Reranking Speech-Recognizer Output

A recognizer proposes several candidate transcripts, and a bigram model must give every plausible candidate a non-zero probability, including word pairs never seen in training.

Why Smoothing Is Required

Smoothing adjusts probability estimates so that unseen but plausible events receive non-zero probability. It redistributes some probability mass from observed events to unseen events. This is important in speech recognition reranking: a candidate transcript should not be assigned absolute impossibility merely because one adjacent word pair was absent from training.

Smoothing methods differ in how they estimate the unseen-event probability and how they use lower-order information. Add-one smoothing is simple but often over-allocates probability to unseen events. Backoff uses a lower-order model when a higher-order event is unavailable. Deleted interpolation combines multiple n-gram orders using weights estimated from held-out evidence.

Measure	Meaning	How to report
Distinct n-grams	Unique observed sequences	Count by order
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count
Perplexity	Exponentiated cross-entropy	Held-out value

Add-One and Add-k Smoothing

For a vocabulary of size V, add-one bigram smoothing can be written as $P_{add1}(w \mid h) = [C(h,w)+1]/[C(h)+V]$. Add-k replaces one with a smaller positive constant k: $P_{addk}(w \mid h) = [C(h,w)+k]/[C(h)+kV]$. The denominator ensures that the conditional probabilities over the defined vocabulary sum to one.

Add-one assigns one pseudo-count to every possible continuation. When V is large and observed counts are limited, the total pseudo-count mass kV can dominate the denominator. This explains why add-one often performs poorly for language modelling. A smaller k reduces the amount of redistributed mass but still requires validation on held-out data.

Measure	Meaning	How to report
Distinct n-grams	Unique observed sequences	Count by order
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count
Perplexity	Exponentiated cross-entropy	Held-out value

Backoff with Discounting

Backoff uses a higher-order probability when sufficient evidence exists and falls back to a lower-order distribution when the higher-order event is absent or discounted. A discount D reduces the count of observed events, reserving probability mass for unseen continuations. A simplified discounted estimate is based on $\max(C(h,w)-D,0)/C(h)$, with the removed mass allocated to a lower-order distribution.

A complete backoff implementation must normalise the remaining mass and define how unseen events receive lower-order probability. It must also handle histories absent from training. The report should state the

discounting formula and any backoff weight calculation. Different discounting schemes produce different results, so the method should not be described only by the word “backoff”.

Measure	Meaning	How to report
Distinct n-grams	Unique observed sequences	Count by order
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count
Perplexity	Exponentiated cross-entropy	Held-out value

Deleted Interpolation

Deleted interpolation estimates weights for unigram, bigram and possibly trigram components by testing how well each order predicts held-out occurrences after the current event is removed from its count estimate. The final conditional probability is a weighted sum such as $\lambda 1\ P(w) + \lambda 2\ P(w|h1) + \lambda 3\ P(w|h2)$, where each λ is non-negative and the weights sum to one.

The weights should be learned from held-out data rather than selected merely because they look reasonable. If the corpus is small, lower-order models may receive substantial weight because higher-order counts are unreliable. As training data grows, the model may place more weight on higher-order estimates when they provide useful predictive information.

Predictions Across Training Sizes

At 10% training data, expect add-one to be comparatively weak because sparse counts and a large vocabulary make its pseudo-count allocation influential. Backoff and interpolation should usually be more robust because they use lower-order evidence. At 50%, higher-order estimates

become more reliable. At 100%, the exact ranking still depends on held-out data and domain match; it must be measured rather than asserted.

For each training proportion, hold out a separate evaluation portion and preserve the same test conditions. Avoid using the test set to estimate interpolation weights. Record the λ values, held-out perplexity, unseen-bigram probability mass and conditional normalisation checks for at least ten histories.

Measure	Meaning	How to report
Distinct n-grams	Unique observed sequences	Count by order
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count
Perplexity	Exponentiated cross-entropy	Held-out value

Probability Normalisation Checks

For every selected history h, calculate the conditional probability for every word in the model vocabulary and sum the results. The total should be approximately one, allowing a small floating-point tolerance such as 1e−8. If the sum differs materially from one, inspect the denominator, vocabulary definition, unknown-word bucket and backoff allocation.

The unseen-bigram mass can be calculated by summing probabilities for all bigrams whose training count is zero, under the same vocabulary definition. The calculation must be explicit: “unseen” may mean unseen in the full training corpus or unseen after a particular history. State which definition is used so that results are comparable.

Measure	Meaning	How to report
Distinct n-grams	Unique observed	Count by order

	sequences	
Unseen rate	Test events absent in training	Percentage
Zero-probability sentences	Sentences with at least one zero event	Count
Perplexity	Exponentiated cross-entropy	Held-out value

Domain Shift in Smoothing

A matching test set resembles the training domain in vocabulary and phrase patterns. An out-of-domain test set may contain different terminology and syntax. An unknown-word edge case deliberately includes words absent from training. All smoothing methods can degrade under domain shift because smoothing cannot invent reliable semantic knowledge about entirely new terms.

Backoff and interpolation may remain more stable than an unsmoothed model because they retain lower-order probabilities. However, an unknown word still needs a defined representation such as <UNK>. A vocabulary threshold can map rare training words to <UNK>, and test words outside the vocabulary can be mapped to the same symbol. The mapping rule must be applied before estimating the model.

Deployment Decision

A reasonable deployment starting point is deleted interpolation or a carefully validated discounted backoff model. Interpolation is attractive when held-out data is available to estimate weights and when multiple n-gram orders contribute useful information. Backoff may be simpler to maintain when the implementation has a well-defined discount and lower-order distribution.

Add-one is useful as a baseline and as a teaching example, but its probability allocation can be inefficient for a large vocabulary. If the

training corpus is very small, a stronger lower-order component may be needed. If the domain changes frequently, vocabulary handling and periodic re-estimation may matter more than a small difference between smoothing formulas.

Q4 – Entropy and Perplexity: Measuring How Good a Language Model Is

Problem Context – Comparing Two Language Models

Two teams report different perplexities for their models. Before accepting either claim, you must understand what entropy and perplexity measure and when two numbers can legitimately be compared.

Entropy

Entropy measures the uncertainty of a discrete probability distribution. For outcomes x with probabilities p(x), entropy in bits is $H(X) = -\Sigma\ p(x)\ \log2\ p(x)$. A uniform distribution over ten symbols has entropy log2(10), approximately 3.322 bits, because all ten outcomes are equally likely.

A skewed distribution has lower entropy than a uniform distribution over the same number of outcomes because probability is concentrated on a smaller effective set of possibilities. Entropy is an intrinsic property of a distribution. Cross-entropy, by contrast, evaluates how well a probability model predicts observations drawn from a data source.

Concept	Expression	Interpretation
Entropy	$-\Sigma\ p\ \log2\ p$	Uncertainty
Cross-entropy	$-(1/N)\Sigma\ \log2\ P(wi\|context)$	Predictive cost
Perplexity	2^H	Effective branching factor

Cross-Entropy of a Language Model

For a test sequence w1,…,wN, the per-word cross-entropy is $H = -(1/N)\ \Sigma\ \log2\ P(wi\ |\ contexti)$. The context depends on the model order. For a unigram it is empty; for a bigram it contains the previous word; for a trigram it contains the previous two words. Sentence-boundary handling and the denominator N must be consistent across models.

If any test event has probability zero, the negative logarithm is infinite. Smoothing or unknown-word handling is therefore necessary for meaningful evaluation. Report whether the count includes sentence boundary tokens, whether unknown words are included, and whether the first positions of sentences are treated consistently.

Concept	Expression	Interpretation
Entropy	$-\Sigma\ p \log_2 p$	Uncertainty
Cross-entropy	$-(1/N)\Sigma \log_2 P(w_i\|context)$	Predictive cost
Perplexity	2^H	Effective branching factor

Perplexity and Branching Factor

Perplexity is $PP = 2^H$ when cross-entropy is measured in bits. It can be interpreted as the weighted average number of equally likely choices the model appears to face at each prediction step. Lower perplexity indicates that the model assigns higher probability to the observed test sequence, under the chosen tokenisation and evaluation setup.

For a uniform distribution over ten symbols, $H = \log_2(10)$ and perplexity is 10. For a skewed distribution, entropy and perplexity are lower. These toy examples provide a numerical check of the formulas. A model's perplexity is not a direct measure of grammaticality, meaning or downstream task success.

Concept	Expression	Interpretation
Entropy	$-\Sigma\ p \log_2 p$	Uncertainty
Cross-entropy	$-(1/N)\Sigma \log_2 P(w_i\|context)$	Predictive cost
Perplexity	2^H	Effective branching factor

Comparing Models Fairly

Two perplexity values can be compared meaningfully only when the evaluation conditions are compatible. Tokenisation, vocabulary, unknown-word mapping, sentence boundaries, test corpus and scoring convention must be aligned. If one model predicts subword units and another predicts words, the number of prediction events differs and the raw values are not directly comparable.

Vocabulary restriction can change perplexity because it changes the probability space and the handling of rare words. Mapping rare words to <UNK> can reduce the number of distinct outcomes but may hide lexical distinctions. Report the vocabulary threshold and unknown-word rate beside perplexity.

Experimental Plan

Train unigram, bigram and trigram models using the best smoothing method selected in Q3. Evaluate on an in-domain text, an out-of-domain text and an edge case such as word-shuffled text or highly repetitive text. For each condition, record cross-entropy, perplexity and the number or proportion of unknown words.

A shuffled text preserves much of the unigram frequency distribution while disrupting local word order. A unigram model may therefore be less affected than a bigram or trigram model. If higher-order models show a substantial change, this indicates that they capture local sequential patterns. It does not prove that they understand sentence meaning.

Interpretation and Limitations

Lower perplexity does not always imply better downstream performance. A speech recogniser, information extractor or chatbot may be evaluated on task-specific accuracy, robustness and user impact. A language model can be well calibrated on common sequences yet fail on rare names or domain-

specific phrases. Therefore, perplexity should be combined with task metrics and error analysis.

The shuffled-text experiment helps identify the kind of information represented by the model. If a model performs poorly when word order is shuffled, it relies on local order. If the unigram score remains similar, that is expected because unigram probabilities ignore order. This is a diagnostic experiment, not a complete test of syntactic competence.

Q5 – Word Classes and Tagset Design

Problem Context – Tagset for a Domain-Specific Corpus

A company wants part-of-speech tags for customer-support chats, which include product names, abbreviations, emoticons and mixed English–Tamil words. The standard English tagsets may not fit this text directly.

Q5 – Open and Closed Word Classes

Open classes readily accept new members. Nouns, lexical verbs, adjectives and many adverbs are open classes because new product names, actions and descriptive terms can enter the language. Closed classes include determiners, pronouns, prepositions, conjunctions and many auxiliary or modal forms. Their membership changes more slowly.

Distributional evidence helps distinguish classes. Nouns can occur after determiners and may take plural or possessive morphology. Verbs can follow subjects and may inflect for tense or agreement. Adjectives can modify nouns or appear after linking verbs. Adverbs can modify verbs, adjectives or clauses. Function words often have grammatical distribution rather than rich lexical meaning.

Tagset design	Strength	Limitation
Coarse	Higher consistency	Less detail
Penn Treebank	Inflectional distinctions	Larger label inventory
Custom	Domain relevance	Needs clear guidelines

English Word Classes in Customer Support

Customer-support chats contain ordinary grammatical words alongside product names, abbreviations, emoticons and code-mixed expressions. A product label may behave like a proper noun even if it contains digits. An abbreviation such as “OTP” functions as a noun in “OTP expired”. A

complaint verb such as “disconnects” is important for extracting the problem being reported.

A standard English tagset may not distinguish every domain-specific category. The design should preserve general grammatical categories while adding only those labels that support a clear annotation decision or downstream need. Excessively fine labels can reduce agreement because annotators must make distinctions that are difficult to observe consistently.

Universal POS Tagset

A coarse Universal POS-style inventory includes categories such as NOUN, VERB, ADJ, ADV, PRON, DET, ADP, AUX, CCONJ, SCONJ, NUM, PART, INTJ, PROPN, PUNCT and SYM. It provides broad cross-linguistic categories and is suitable for general grammatical analysis. It is relatively compact and can support higher annotation agreement.

For customer-support extraction, PROPN can help identify product and service names, while VERB can identify complaint actions. However, a general POS tag does not always identify the boundary of a multiword product name or distinguish a product code from an ordinary proper noun. Named-entity annotation may be needed in addition to POS tagging.

Tagset design	Strength	Limitation
Coarse	Higher consistency	Less detail
Penn Treebank	Inflectional distinctions	Larger label inventory
Custom	Domain relevance	Needs clear guidelines

Penn Treebank Tagset

The Penn Treebank tagset is more detailed than a coarse universal inventory. It distinguishes noun forms such as NN, NNS, NNP and NNPS; verb forms such as VB, VBD, VBG, VBN, VBP and VBZ; and several determiner, pronoun, adverb and preposition categories. This granularity represents useful inflectional and syntactic distinctions.

The benefit is richer grammatical information. The cost is a larger label space and more opportunities for disagreement. Distinctions such as VBG versus NN can be difficult when a word ending in -ing is used nominally or verbally. The annotation guide must provide context-sensitive examples and tie-breaking rules.

Tagset design	Strength	Limitation
Coarse	Higher consistency	Less detail
Penn Treebank	Inflectional distinctions	Larger label inventory
Custom	Domain relevance	Needs clear guidelines

Custom Tagset for Support Chats

A possible custom tagset can retain broad POS categories and add domain labels: PROD for product or service names, ABBR for domain abbreviations, CHATEMO for emoticons, CODE for alphanumeric product or error identifiers, MIXED for code-mixed lexical forms whose grammatical role cannot be assigned confidently, and COMPLAINT_VERB as a task-specific verb subtype.

Each additional tag must have a clear definition and examples. PROD should identify lexical spans that name a product or service, not every capitalised word. ABBR should cover conventional abbreviations such as OTP when used as abbreviations. CODE should be reserved for identifiers such as error codes. COMPLAINT_VERB should be assigned only to verbs expressing a fault or negative service action, not all verbs.

Tagset design	Strength	Limitation
Coarse	Higher consistency	Less detail
Penn Treebank	Inflectional distinctions	Larger label inventory
Custom	Domain relevance	Needs clear guidelines

Granularity, Agreement and Ambiguity

A coarse tagset often improves agreement because annotators make fewer fine distinctions. It may, however, lose information needed for detailed extraction. A fine tagset can represent domain-specific differences but may increase ambiguity and lower agreement. Accuracy comparisons across tagsets must account for the fact that the label inventories are different.

Cohen's kappa measures agreement beyond chance: $\kappa = (p_o - p_e)/(1 - p_e)$, where p_o is observed agreement and p_e is expected agreement from the annotators' marginal label distributions. Report the annotation unit, treatment of punctuation, and whether agreement is calculated token by token or at span level.

Tagset design	Strength	Limitation
Coarse	Higher consistency	Less detail
Penn Treebank	Inflectional distinctions	Larger label inventory
Custom	Domain relevance	Needs clear guidelines

Annotation Procedure

Select at least 100 sentences from the chosen support-chat corpus. Prepare annotation guidelines before labelling. Annotate each token with the three tagsets. After a time gap, re-annotate independently without viewing the original labels, or ask a second annotator to label the same sample. Keep the two annotation files separate until agreement is calculated.

For each tagset, report Cohen's kappa, percentage of ambiguous word types, average number of tags per type and five most-confused tag pairs. A confusion matrix helps identify systematic disagreements. If only one annotator is available, self re-annotation is weaker evidence than independent peer annotation and should be disclosed as such.

Design Decision

For information extraction that must find product names and complaint verbs, a custom tagset can be useful if its additional labels are reliably annotated. A practical design is to retain general POS labels while adding a product-name label and a complaint-verb subtype. If agreement for custom tags is poor, use a coarse POS tagset plus a separate named-entity or complaint-span layer.

This layered design avoids forcing every task-specific distinction into the POS inventory. It also allows a standard POS tagger to be reused while domain-specific extraction is handled by a separate component. The final choice should be justified by measured kappa and ambiguity rates rather than by the apparent sophistication of the tagset.

Q6 – Rule-Based, Stochastic and Transformation-Based Tagging

Problem Context – Tagging News Headlines and Social Posts

Headlines such as “Time flies”, “Saw the saw” or “They can fish” are ambiguous. You must tag text accurately using three different approaches and compare them.

Rule-Based Tagging

A rule-based tagger uses a lexicon and hand-written contextual constraints. The lexicon stores possible tags for known words. Rules use nearby words, suffixes, capitalisation and sentence position to resolve ambiguity. For example, a determiner before a word supports a noun analysis, while a modal auxiliary before a base-form verb supports a verb analysis.

Rule-based systems are transparent because a developer can inspect the lexicon and each rule. They can work with limited labelled data if linguistic knowledge is available. However, rules may be brittle, incomplete and difficult to maintain when the domain changes. Ambiguous words such as “flies”, “saw” and “can” require context.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Rule-Based Constraints

A practical rule set may include: after a determiner, prefer a noun or adjective; after a modal, prefer a base-form verb; after “to” in an infinitival construction, prefer a base verb; after a possessive determiner, prefer a noun; before a noun, allow an adjective; after a linking verb, allow an adjective; sentence-initial capitalisation may support a proper noun; a word

ending in -ly may be an adverb; a plural -s form may be a plural noun or third-person verb; and punctuation can constrain sentence boundaries.

These are tendencies, not exceptionless grammatical laws. For example, “fast” can be an adjective or adverb, and “daily” can be an adjective or adverb. Rules should be ordered or assigned priorities, and the evaluation should record which rules changed a baseline tag. A lexicon should retain multiple possible tags for ambiguous forms.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Most-Frequent-Tag Baseline

The most-frequent-tag baseline assigns each known word its most common tag in the training corpus. Unknown words receive a fallback tag, often inferred from suffixes or assigned a default noun tag. This baseline is important because a complex tagger should be compared with a simple, reproducible reference point.

The baseline can perform surprisingly well on frequent known words because many word forms have a dominant tag. It generally struggles with ambiguous words in unusual contexts and with unknown words. Report known-word and unknown-word accuracy separately so that overall accuracy does not conceal the weakness of the unknown-word strategy.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost

Pruning redesign	Lower storage	May discard rare useful events

Stochastic Bigram HMM Tagger

A hidden Markov model treats POS tags as hidden states and observed words as emissions. The tag sequence probability is approximated using transition probabilities P(tag_i | tag_(i−1)) and emission probabilities P(word_i | tag_i). The most likely tag sequence is found with Viterbi decoding. Start and end states should be included in the implementation.

For unknown words, a system may use an UNK emission, suffix-based probabilities, or a small set of word-shape features. The treatment must be stated. Without unknown-word handling, unseen words may receive zero emission probability and make every candidate path impossible. Smoothing transition and emission probabilities is also necessary.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Viterbi Decoding and Complexity

Viterbi decoding uses dynamic programming to retain the best path ending in each tag at each word position. For sentence length T and tagset size K, a straightforward bigram HMM evaluates approximately T×K×K transitions, giving time complexity $O(TK^2)$. Memory can be O(TK) when backpointers are stored for the full sentence.

The transition count grows linearly with sentence length and quadratically with the tagset size. This makes tagset granularity a computational consideration as well as an annotation consideration. Sparse transition

representations can avoid evaluating tag pairs that are impossible or never observed, provided the model still handles unseen transitions correctly.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Transformation-Based (Brill) Tagging

A Brill-style tagger begins with an initial tagging, commonly the most-frequent-tag baseline, and learns ordered transformations that correct errors. Templates describe candidate contexts, such as changing a tag when the previous word has a particular tag or when the current word has a particular suffix. The learner selects transformations that improve training accuracy according to a scoring function.

The ordered rule list is interpretable: each transformation states when a tag should change. The method can correct systematic errors without requiring a large neural model. However, candidate rule generation and scoring may become expensive when the corpus, tagset or template inventory grows. The stopping threshold and maximum number of rules should be documented.

Prediction and Evaluation

Before running the experiment, predict that the most-frequent-tag baseline will be competitive on known frequent words but weak on ambiguous contexts and unknown words. Rule-based tagging may improve selected constructions and remain interpretable. The HMM may capture sequential tag patterns, while Brill tagging may correct systematic baseline errors. The actual ranking can change with corpus size and domain match.

Use a fixed tagged corpus split and seed. Report overall, known-word and unknown-word accuracy. Include a confusion matrix for frequent errors, the number of Brill transformations and accuracy gain after each transformation, and the number of Viterbi transitions evaluated. Repeat with 10%, 50% and 100% of the training data.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Comparing Taggers and Low-Resource Recommendation

All three approaches may share errors caused by ambiguous words, annotation inconsistencies, domain-specific vocabulary and insufficient context. With little data, a transparent rule-based component can encode reliable linguistic knowledge, while a baseline provides a stable starting point. An HMM may be limited by sparse transition and emission estimates. Brill learning may overfit if candidate rules are selected from a very small sample.

For a low-resource language with approximately 2,000 tagged sentences, a hybrid of a simple statistical baseline and carefully validated linguistic rules is a defensible starting recommendation. It should be tested against the baseline and expanded only when evidence shows that each added component improves held-out accuracy. The recommendation should not be presented as a measured result unless the experiment has been run.

Q7 – Diagnose, Redesign and Validate

Review the evidence you generated while working on Q1–Q6.

Select ONE solution that demonstrates a significant limitation as its problem size oroperating requirement increases.

Diagnose the Bottleneck

The design challenge requires selecting one limitation demonstrated by the earlier experiments. A suitable example is the growth of n-gram tables as n and corpus size increase. Higher-order n-grams create many distinct contexts, and a large proportion may occur only once. These entries consume memory but provide weak estimates for future text. The diagnosis should use measured table entries, singleton counts and unseen test rates.

The cause is the combination of a large vocabulary, longer contexts and limited repeated observations. The system stores exact sequences even when their counts are too small to support reliable prediction. This is not merely a slow implementation; it is a statistical data sparsity problem. A redesign should target the source of wasted representation rather than simply replacing the model with a more advanced architecture.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Objective, Redesign and Validation

A measurable objective could be: reduce stored n-gram entries by at least a chosen target percentage while keeping held-out perplexity within an agreed tolerance, or reduce the proportion of singleton entries without materially

increasing unseen test events. The target must be selected before the redesign is implemented. A possible strategy is frequency-threshold pruning: retain frequent higher-order n-grams and back off rare contexts to lower-order distributions.

The revised data flow is: tokenise and normalise; count n-grams; retain entries meeting a minimum frequency; calculate discounted probability mass for retained entries; use lower-order backoff for pruned or unseen contexts; evaluate on the same held-out test set. This avoids storing every rare high-order event. It introduces a threshold-selection trade-off: a high threshold saves memory but may discard useful phrases.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Pseudocode and Trade-offs

Pseudocode: initialise n-gram counters; count all training n-grams; for each n-gram, if count is at least threshold[n], store it, otherwise add its discounted mass to the backoff bucket; estimate lower-order probabilities; normalise each history; evaluate test perplexity and storage entries. The same preprocessing, vocabulary and test set must be used for the original and redesigned systems.

The redesign may improve memory use and reduce unreliable estimates, but it can lose rare yet meaningful expressions such as product names or error messages. A domain-specific exception list or named-entity store may preserve important rare terms. Additional preprocessing and threshold tuning introduce complexity. The original model may remain preferable for

a small corpus where storage is not a concern and rare sequences are important.

Approach	Useful property	Main risk
Rule-based	Transparent constraints	Brittle rules
HMM + Viterbi	Sequence probabilities	Sparse estimates
Brill	Ordered corrections	Rule-search cost
Pruning redesign	Lower storage	May discard rare useful events

Validation and Final Defence

Compare original and redesigned systems using counted measures: number of stored n-gram entries, number of singleton entries, unseen test n-gram rate, zero-probability sentences if relevant, and held-out perplexity. Runtime can be reported as supporting evidence but should not be the only basis for success. Repeat the comparison under in-domain and out-of-domain conditions to expose robustness limits.

The prediction is supported only if the measured results show the expected trade-off. If memory entries fall but perplexity worsens sharply, the threshold may be too high. If the change is small, the selected corpus may not contain enough rare contexts for pruning to matter. A redesign that fails to outperform the original can still be informative when the hypothesis and failure are analysed honestly.

Overall Comparison Across the Assignment

The seven questions are connected by a common theme: representation choices determine what a model can learn. Tokenisation defines the units; n-gram order defines the context; smoothing determines how probability is shared; entropy and perplexity measure predictive uncertainty; tagsets define grammatical categories; and tagging algorithms use those categories

to infer structure. The design challenge then asks how to improve a component when evidence exposes a limitation.

A fair comparison requires consistent data preparation and explicit assumptions. A model should not be judged by a single metric. Coverage, accuracy, memory, computational work, interpretability and robustness may all matter. The best system is the one that satisfies the intended task under its actual data and resource constraints.

Limitations and Threats to Validity

Corpus selection can bias conclusions. Customer-support messages may contain repeated templates, spelling variation and domain-specific terms. A news corpus may have different sentence lengths and vocabulary. Annotation agreement can be affected by unclear guidelines or annotator familiarity. Perplexity can be distorted by tokenisation and unknown-word policies. These limitations should be stated alongside the results.

A fixed random seed improves reproducibility but does not remove sampling uncertainty. Repeating the experiment with more than one split can reveal whether a result is stable. If only one split is required by the assignment, state that the result is conditional on that split. Avoid broad claims about all languages or all domains based on one dataset.

Final Conclusion

This assignment demonstrates that NLP design requires more than selecting a named algorithm. Tokenisation is a policy choice; unsmoothed n-grams suffer from sparse observations; smoothing balances observed and unseen events; entropy and perplexity quantify predictive uncertainty; tagset granularity affects both information and agreement; and tagging methods trade transparency, data requirements and computational cost.

The most defensible conclusions are those supported by reproducible counts and carefully controlled evaluation. The student should replace all illustrative examples with actual corpus outputs, attach source code and

CSV files, and be prepared to explain the assumptions and limitations in a viva. The overall engineering process is to predict, implement, measure, analyse, redesign and validate.

References and Corpus Documentation

Use the exact corpus reference for every dataset used. Suitable resources to investigate include the Brown Corpus for general English text, the Penn Treebank for English POS annotation, Universal Dependencies treebanks for cross-linguistic annotation, and a documented customer-review or support-chat dataset for the domain experiment. Confirm the licence, version, preprocessing and citation instructions from the dataset provider before use.

For theoretical background, consult standard NLP textbooks and the official documentation for the libraries used. Record the edition and publication details in the final bibliography. Do not cite a corpus merely because it is mentioned in the assignment prompt; cite the actual resource downloaded and used. The final report should include access dates for online resources where required by the institution.

APPENDIX A – REPRODUCIBLE PYTHON STARTER CODE

The following compact starter code demonstrates the core counting operations for token statistics and n-gram coverage. It is a template: adapt the tokenizer and data-loading step to the selected corpus, run it locally, and save the resulting CSV files. The code does not claim to be a complete implementation of every smoothing and tagging method.

```
import re
from collections import Counter


def whitespace_tokens(text):
    return text.split()
```

```
def punctuation_tokens(text):
    return re.findall(
        r"[A-Za-z]+(?:'[A-Za-z]+)?|\d+(?:[./:]\d+)*|[^\w\s]",
        text.lower()
    )


def refined_tokens(text):
    text = text.lower()
    pattern = r"[a-z]+(?:-[a-z]+)+|[a-z]+(?:'[a-z]+)?|\d+(?:[./:]\d+)*|4g/5g|:\)|[^\w\s]"
    return re.findall(pattern, text)


def token_statistics(tokens):
    counts = Counter(tokens)
    n = len(tokens)
    v = len(counts)
    hapax = sum(1 for count in counts.values() if count == 1)

    return {
        "N": n,
        "V": v,
```

```
        "TTR": v / n if n else 0,
        "hapax": hapax,
        "top10": counts.most_common(10)
    }


def ngrams(tokens, n):
    return [
        tuple(tokens[i:i+n])
        for i in range(len(tokens) - n + 1)
    ]


def ngram_coverage(train_tokens, test_tokens, n):
    train = set(ngrams(train_tokens, n))
    test = ngrams(test_tokens, n)

    unseen = sum(1 for item in test if item not in train)

    return {
        "distinct_train": len(train),
        "test_events": len(test),
        "unseen_events": unseen,
        "unseen_percent": 100 * unseen / len(test) if test else 0
    }
```

```
text = """Natural Language Processing helps computers understand human language.
Machine learning is used in NLP applications."""

tokens = refined_tokens(text)

print("Tokens:", tokens)
print("Token Statistics:", token_statistics(tokens))

print("Unigrams:", ngrams(tokens, 1))
print("Bigrams:", ngrams(tokens, 2))
print("Trigrams:", ngrams(tokens, 3))

train_tokens = refined_tokens(
    "Natural language processing uses machine learning."
)

test_tokens = refined_tokens(
    "Natural language processing uses deep learning."
)

print("Bigram Coverage:", ngram_coverage(train_tokens, test_tokens, 2))
print("Trigram Coverage:", ngram_coverage(train_tokens, test_tokens, 3))
```